



Ariadne on a single H100

Results report: cluster setup, capacity, benchmarks, queue probes and agent validation

App	Ariadne, a read-only root-cause agent for Kubernetes (Track B). It names the object that has to change and cites only evidence it was shown, or answers <code>inconclusive</code> .
Model	Qwen3.8-27B in FP8 on vLLM 0.29.0, one worker per HAMi half
Cluster	One NVIDIA H100 80 GB on Lambda Cloud, single-node k3s, split by HAMi into two halves of 39 GiB and 50% of the compute
Gateway	Go gateway that guards, admits, places and queues every call, with an opt-in Mooncake KV hop and KEDA scaling of the vLLM pool
Agent workload	A LangGraph agent with 11 read-only tools, 5 to 16 chained model calls per run, replaying a golden set of 26 faults recorded from a real cluster
Measured	27 September to 8 October 2026, across four GPU sessions (A100, H100 PCIe, H100 SXM5)
Data	<code>metrics/</code> (golden rows and summaries, Prometheus scrapes, time series), <code>report/report.ipynb</code> , <code>design/findings.md</code> (F1 to F47), <code>design/decisions.md</code> (D-1 to D-51)
Repository	<code>github.com/cduggn/Ariadne</code>

Contents

1. Executive summary

2. Setup overview

3. Part 1. Cluster setup

4. Part 2. Capacity on paper and measured

5. Part 3. Benchmarks: routing, the knee, admission

6. Part 4. Who waits where: the queue probes
7. Part 5. Scaling the worker pool

8. Part 6. Validation with agent runs

9. Part 7. Operating it: alerts, 10× traffic, open questions

10. Exhibits. Grafana screenshots

11. Appendix. Reproducing the runs

Executive summary

Ariadne is an agent that diagnoses Kubernetes faults by calling read-only tools, and the system around it is a self-hosted inference stack: a Go gateway in front of two vLLM workers that share one H100 through HAMi. Cluster data is restricted, so it never leaves the team's own GPUs. Every number below comes from a committed file in `metrics/`, and each carries its finding number (F) from `design/findings.md`.

- **The model is the biggest lever on answers.** Qwen3.8-27B diagnoses 86.5% of the golden set [95% CI 74.7–93.3], against 51.9% [38.7–64.9] for Qwen3-8B. It scores 100% on red herrings, the tier where a rule or dashboard points at the wrong object (F1).
- **KV cache is the first limit.** One H100 half holds 51,092 tokens of KV, about two full-length (24,576-token) runs. The fit calculator was 34% optimistic for this hybrid model, though within 1% for the standard-attention 8B (F5, F6).
- **The knee is between 8 and 16 concurrent agent runs.** With 52 runs per level the pass rate holds at 83–87% up to 8, then falls to 63.5% at 16 and 23.1% at 32 (F24, F40).
- **Sizing admission to KV moves the waiting from vLLM to the gateway.** An in-flight cap of 4 per worker cut vLLM preemptions by about 95% on the same node and lowered tail latency about 15%, but finished fewer runs at 16 and 32, because queued requests hit their deadline (F36). Across the full sweep vLLM's own queue never held more than 2 requests (F41).
- **Keeping a run on its worker saves prefill without changing answers.** Without stickiness vLLM recomputed 10% more prompt tokens and steps were 8–11% slower, at the same pass rate (F29). The prefix cache serves 87.8% of prompt tokens (F9).
- **Recovery and scaling are measured in minutes.** A deleted worker was serving again in 5 min 37 s (F44). KEDA asks for a second worker about a minute after a capacity shed, but a new worker takes 8 min 21 s to load, so scale-down waits 15 quiet minutes (F39, D-51).
- **Two things are unconfirmed.** The Mooncake KV hop completed its protocol 53 times with no failures, but nobody has confirmed that the destination pulled the KV rather than recomputing it (F35, F45). vLLM's abort counter didn't move when clients left, so freeing their KV is also unconfirmed (F43).

How the evaluation works. Every measured run replays the golden set: 26 faults recorded from a real Kubernetes cluster, in four tiers (easy, multi-hop, red herring, right-sizing). The model and its tool calls are real and go through the gateway; only the tool answers come from the recordings, so every run sees identical evidence and the pass rates are comparable. The same agent reads a live cluster through a read-only kubectl, but no live-cluster run was measured.

Setup overview

Item	Value
Model	Qwen3.8-27B, FP8 weights (28.8 GiB). A hybrid model: Gated DeltaNet layers keep a fixed recurrent state, and only its full-attention layers hold KV (64 KiB per token)
Engine	vLLM 0.29.0 (<code>vllm-openai:v0.29.0-cu129</code> , pinned by digest). v0.30.0 crash-loops on start (F19)
Engine settings	context 24,576 tokens; at most 32 sequences; 8,192 batched tokens per step (chunked prefill); prefix caching on; <code>gpu_memory_utilization</code> 0.90 of the slice
Sampling	Qwen3 non-thinking: temperature 0.7, top_p 0.8, top_k 20; at most 768 output tokens per call; <code>tool_choice</code> auto (D-38)
Agent	LangGraph state graph over LangChain tools; 11 read-only tools; step caps of 16 (investigate), 30 (audit) and 12 (right-size); stops before a 23,000-token context
Golden set	26 tasks: 14 easy, 7 multi-hop, 4 red herring, 1 right-sizing. Scored deterministically by <code>evals/checker.py</code> (v2, D-41)
Access path	Agent on the laptop → SSH tunnel → gateway Service on the node → vLLM worker. Every call carries <code>X-Request-Id</code> , <code>X-Tenant</code> , <code>X-Priority</code> and <code>X-Data-Class: restricted</code>

Part 1. Cluster setup

Everything runs on one Lambda Cloud instance with a single H100 80 GB, managed as a one-node k3s cluster. `make up` takes the first GPU with capacity (GH200, then H100, then A100), and one cloud-init detects the card and deploys the matching topology (F18). HAMi splits the card into two slices so two vLLM workers share it. Prometheus scrapes every component, and Grafana shows three dashboards.

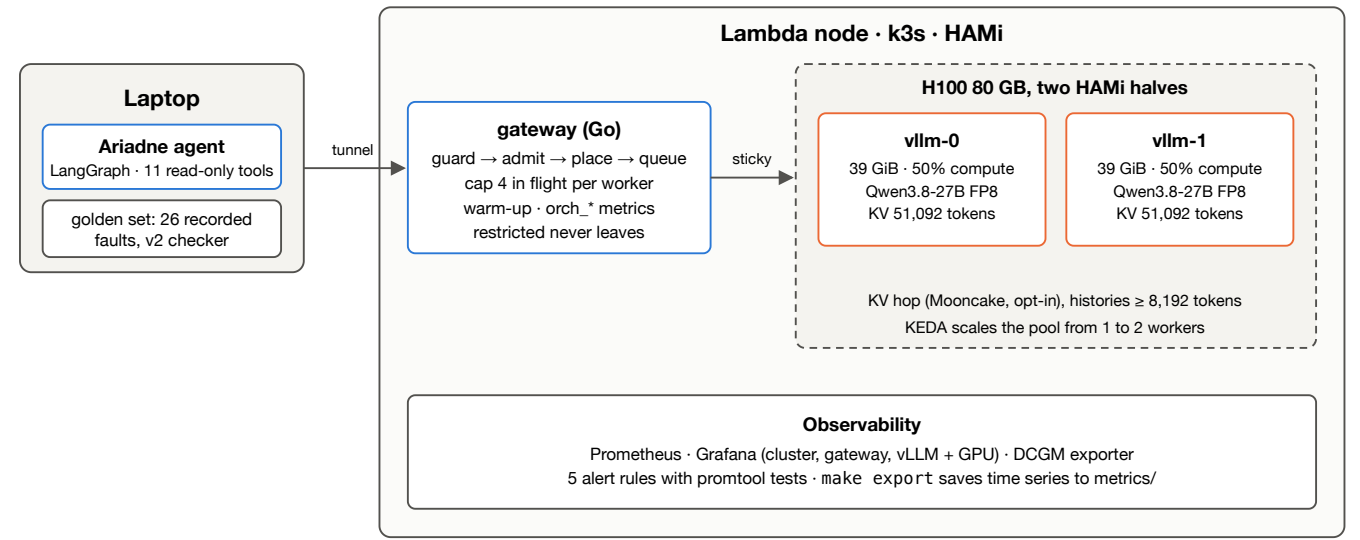


Figure 1. The agent runs on the laptop and reaches the gateway through an SSH tunnel. The gateway places each call on one of two vLLM workers that share the H100 through HAMi.

As deployed

Pod	Role	GPU memory	Compute	KV cache (tokens)	24,576-token runs that fit
vllm-0	prefill and decode	39 GiB	50%	51,092	2.08
vllm-1	prefill and decode	39 GiB	50%	51,092	2.08
Total		78 of 80 GB	100%	102,184	4.2

Table 1. The inference pods, their HAMi slices, and the KV pool vLLM reported at start-up (`metrics/kv-qwen3.8-27b-fp8-h100-half-20261006-125054.log`). Each worker does its own prefill and decode; there is no prefill/decode split.

Component	Deployment	Configuration
Gateway	1 pod, Go, public image pinned by digest	Guard (400), tenant token quota (429), KV and queue deadline (503); in-flight cap 4 per worker (D-43); priority queue per worker with deadlines of 10 s (interactive) and 30 s (batch); <code>prefix_then_load</code> placement; warm-up probes before a worker takes traffic
KV hop	vLLM's MooncakeConnector, gateway flag	Moves a run's KV to the other worker when its history is at least 8,192 tokens and the copy is cheaper than recomputing it (D-42)
KEDA	ScaledObject on <code>statefulset/vllm</code>	1 to 2 workers; demand trigger (<code>in flight + queued ÷ 4</code>) and a capacity-shed trigger; 15-minute scale-down window (D-49, D-51)
Observability	kube-prometheus-stack, Grafana, DCGM exporter	Scrapes the gateway, both workers and the GPU; three dashboards; five alert rules (D-45)
Agent	laptop, Python 3.12, <code>python -m agent</code>	Read-only by construction: a <code>kubectl</code> verb allowlist, no Secrets, no exec; LangSmith tracing forced off

Table 2. Other components.

Request path

1. The agent sends an OpenAI chat completion with its headers. The run id ties the 5 to 16 calls of one diagnosis together.
2. **Guard.** Malformed bodies, streaming and client-supplied `kv_transfer_params` get a 400.
3. **Admit.** A tenant over its token quota gets a 429. A request whose worker is below 20% free KV, or that would miss its queue deadline, gets a 503 with its reason (`kv_free`, `timeout_queue`) and a `Retry-After`. Only a 503 may leave the box, and a restricted request never does (fuzz test and alert).
4. **Place.** A run stays on the worker that holds its history, unless that worker is much busier or has restarted.
5. **Queue.** At most 4 requests run per worker. The rest wait in priority order; batch audits are shed before interactive investigations.
6. The agent waits out a 429 or 503 (and, since 10-08, a 502 from a worker that went away) for up to about 15 s before the run fails (D-44).

Deployment notes

Issue	Cause	Resolution
vLLM v0.30.0 crash-loops at start	The image ships a cu130 torch with a cu129 torchvision (vllm#59157)	Stay on v0.29.0, pinned by digest (F19)
Agent's tool calls collapsed into whitespace (0/5)	<code>tool_choice="required"</code> made vLLM constrain decoding with a grammar that degenerated	<code>tool_choice="auto"</code> with the model card's sampling; the agent nudges a reply with no tool call (D-38)
Malformed tool-call arguments crashed a run	The model sometimes sent a JSON string holding a Python dict	Turned into an invalid tool call the model is told about; the run continues (D-39)
Three billed nodes after a slow boot	<code>make up</code> retried a launch that had already created the instance	<code>make up</code> refuses while a node exists; <code>make resume</code> finishes a slow boot (F20, F21)
Overflow backend unavailable	The provider returned HTTP 402 for every call despite a credit grant	Overflow decided but left null; restricted traffic never overflows anyway (F22)
Sweep rows for Qwen3.8 written to one file	<code>Path.with_suffix</code> cut the tag at the dot in "qwen3.8"	Fixed in the runner, with a test; summaries were unaffected (F37)

Table 3. Problems met on the way and how each was resolved.

Part 2. Capacity on paper and measured

`serving/fit.py` predicts, for every model and topology, the weights, the KV bytes per token, the KV pool, and how many requests fit at full length and at the app's length. vLLM then reports the pool it actually allocated. A gate refuses to deploy a pairing that can't hold one full-length request.

Model, where	Weights	KV per token	Paper pool	Measured	Gap
Qwen3-8B-AWQ, A100 20 GiB slice	5.7 GiB	144 KiB	79,699	79,056	-0.8%
Qwen3-30B-A3B AWQ, whole A100	16.9 GiB	96 KiB	197,563	185,136	-6.3%
Qwen3.8-27B-FP8, whole H100	28.8 GiB	64 KiB	653,004	525,797	-19.5%
Qwen3.8-27B-FP8, H100 39 GiB half	28.8 GiB	64 KiB	77,926	51,092	-34.4%

Table 4. KV pool in tokens, paper against vLLM's own start-up report. Qwen3.8 also keeps about 147 MiB of recurrent state per sequence, which the token count doesn't show.

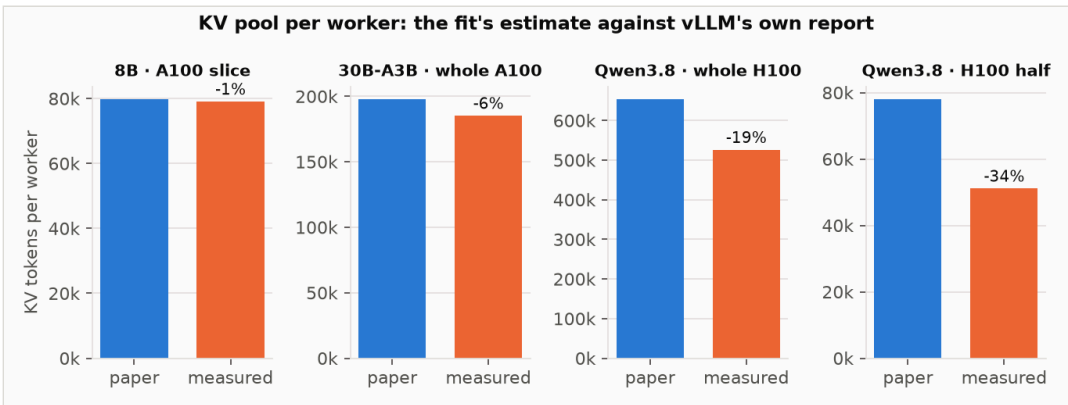


Figure 2. The fit is accurate for standard attention and over-optimistic for the hybrid model. The fixed costs it underestimates (recurrent state, the vision encoder and multi-token-prediction head in the checkpoint, larger activations) weigh more on a smaller slice (F5; the cause is unverified).

What that means for concurrency

- **About two full-length runs per half.** vLLM reports a maximum concurrency of 2.08 at 24,576 tokens, against the fit's 2.9. KV, not the 32-sequence cap, is the first limiter (F6).
- **An agent run is shorter than the maximum on average, but grows.** Each run starts at about 4.7k tokens and adds a few hundred per step. Audits reach about 15k tokens.
- **So the gateway allows 4 in flight per worker.** D-43 sizes the cap from the measured pool: four runs at the app's length of about 12k tokens, with the shared prefix cached once, need about 36k of the 51k tokens. The earlier cap of 16 was sized for the 8B on an A100 slice and let more work into vLLM than its KV holds (F26).

Shared and unique tokens

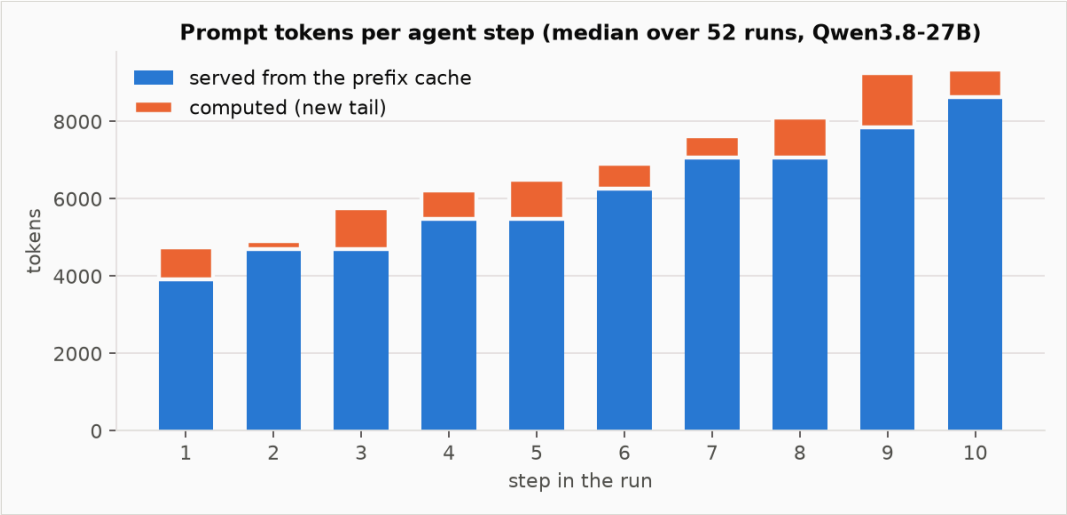


Figure 3. Every call starts with the same prefix (ruleset, tool schemas, cluster card), now about 3.9k tokens (F47). Each step adds a few hundred tokens. By step 10 the median prompt is 9,337 tokens, and vLLM computes only 713 of them. Across the golden run, 87.8% of prompt tokens came from the prefix cache. That is lower than the 8B's 94.9% because the hybrid layers can only resume from block-boundary checkpoints (F8–F10, F9's cause unverified).

Part 3. Benchmarks: routing, the knee, admission

All benchmarks are the agent itself: the golden set at a fixed number of concurrent runs, through the gateway, each run making its chained calls. Pass rates are v2 with 95% Wilson intervals.

3.1 Placement: does keeping a run on its worker matter? (10-06)

	prefix_then_load (sticky)	least_loaded
v2 pass [95% CI]	86.5% [74.7–93.3]	88.5% [77.0–94.6]
Cached share of prompt	87.8%	85.6%
Uncached prompt tokens prefilled	416,445	459,909 (+10.4%)
Step latency p50	2.86 s	3.10 s (+8%)
Step latency p95, steps after the first	25.3 s	28.1 s (+11%)
Continuing steps back on their run's worker	94%	77%

Table 5. The routing A/B: the golden set twice per arm at 4 concurrent runs, only the pick policy changed (F29).

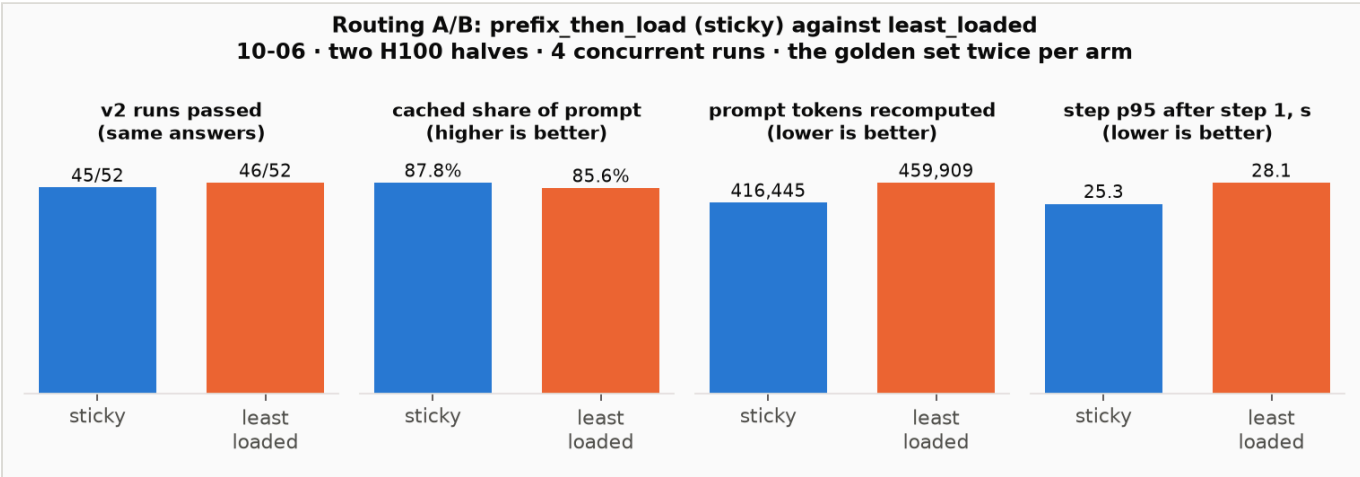


Figure 4. Stickiness saves prefill and time without changing answers. The gap is modest at this load, because least_loaded picks the run's previous worker 77% of the time anyway and both workers hold the shared prefix from warm-up. We expect it to widen under uneven or heavier load, which no run has tested (F29).

3.2 Under load: the knee

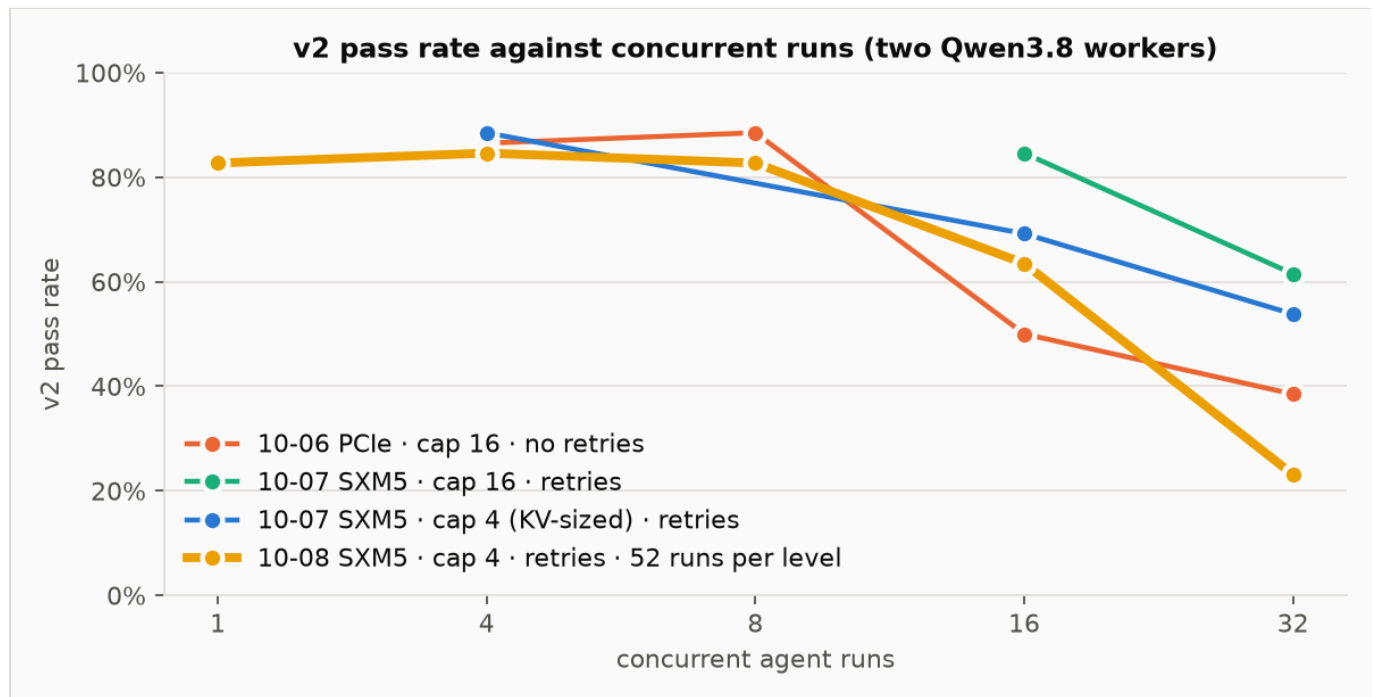


Figure 5. The pass rate against concurrent agent runs for four configurations. The thick line is the 10-08 full sweep with 52 runs per level, the most robust of the four (F24, F36, F40).

Concurrent runs	v2 pass [95% CI]	Refused by the gateway
1	82.7% [70.3–90.6]	0
4	84.6% [72.5–92.0]	0
8	82.7% [70.3–90.6]	kv_free 1
16	63.5% [49.9–75.2]	timeout_queue 127, kv_free 38
32	23.1% [13.7–36.1]	timeout_queue 220, kv_free 33

Table 6. The 10-08 sweep on an H100 SXM5: cap 4, refusal retries, 52 runs per level. The 8B on an A100 stayed at 48–56% at every level, so its limit is the model, not capacity (F40).

- **KV is the limiter.** Each half holds about two full contexts; 16 runs put about 8 per worker at 5–15k tokens each (F24).
- **At 32, most runs fail on the queue deadline.** Requests wait in the gateway's queue past their deadline, and that wait outlasts the agent's ~15 s of retries (F34, F40).

3.3 Shed at the door, or preempt in the engine? (10-07, same node)

	Cap 4 (sized to KV)	Cap 16 (old)
v2 pass at 16 / 32 concurrent runs	69.2% / 53.8%	84.6% / 61.5%
vLLM preemptions added (16 / 32)	1 / 2	9 / 34
Cached share of prompt (16 / 32)	81.3% / 81.6%	73.3% / 72.7%
Step latency p95 (16 / 32)	21.8 s / 21.7 s	25.3 s / 26.9 s
Refusals	timeout_queue 42 / 83	kv_free 58 / 132

Table 7. Same node, both with retries. Sizing admission to KV keeps the engine healthy (about 95% fewer preemptions, 8 points more cache, about 15% lower tail latency) but finishes fewer runs, because queued requests hit their deadline before the retries run

out. The pass-rate intervals overlap at 26 runs per level, so the completion gap is suggestive. The next step is a cap of 6–8, or a longer interactive deadline (F36).

3.4 Inside the engine

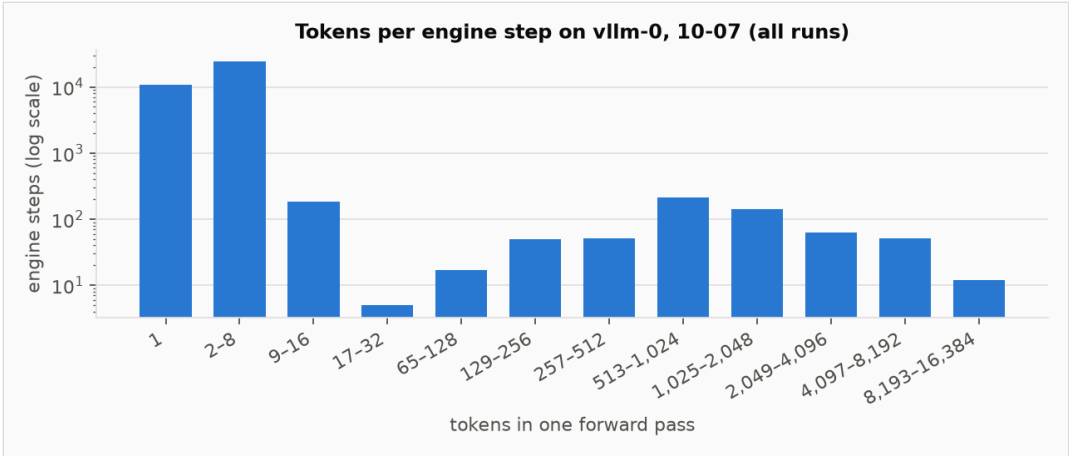


Figure 6. Continuous batching appears only when requests overlap: of 44,121 engine steps on one worker, 31,169 processed a single token. Chunked prefill is configured but rarely binds; only 8 steps fell in the 4,097–8,192-token range, because the prefix cache keeps each step’s uncached tail to a few hundred tokens (F14, F15).

Part 4. Who waits where: the queue probes (10-08)

`make bench` ends with three probes against a golden run at 4 concurrent runs, then `make export` saves the node's Prometheus history for the whole window. The probes are: three 10.5k-token batch prompts, three clients that disconnect after 3 s, and deleting one worker. The marks on each chart are the probe times.

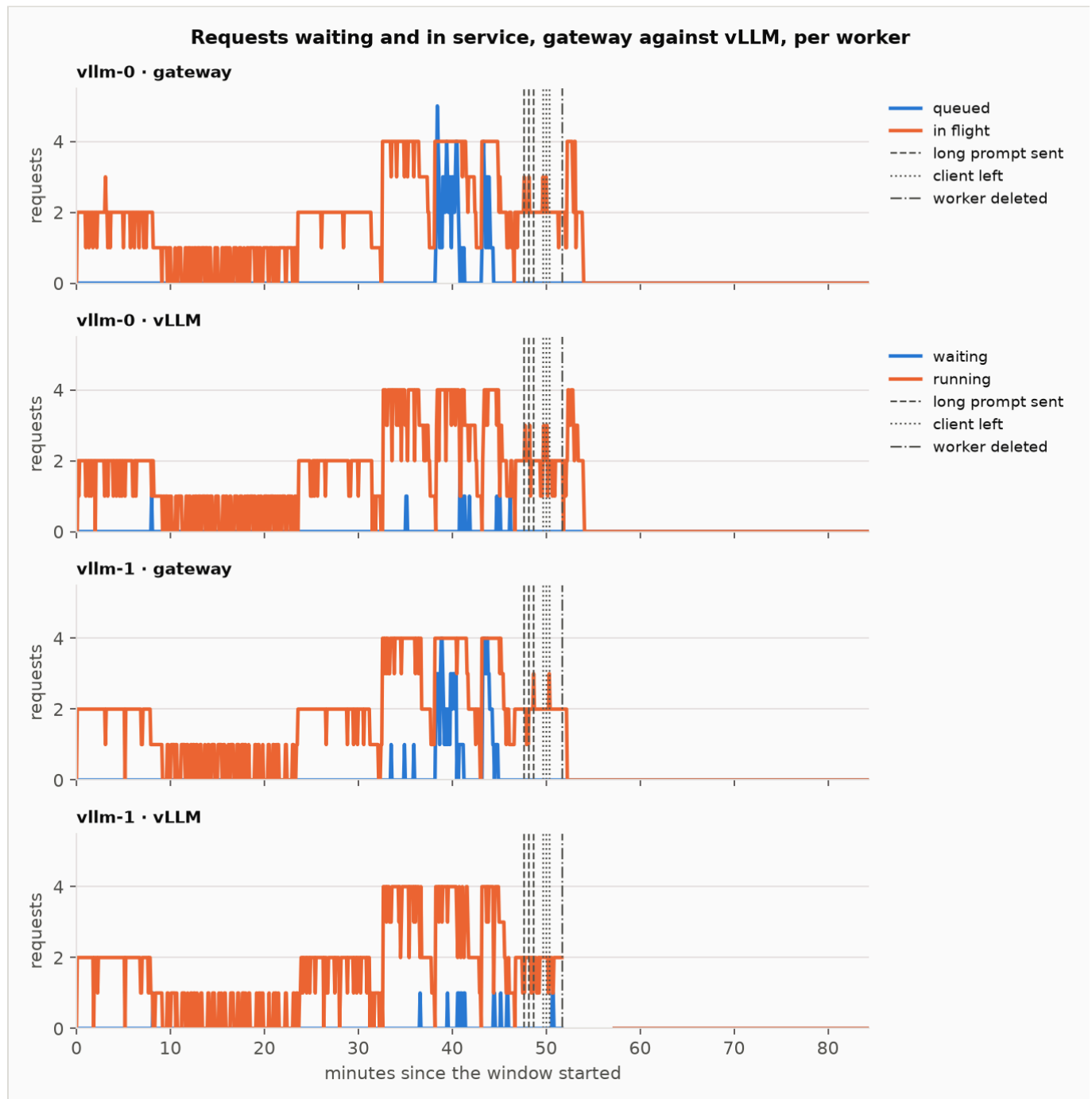


Figure 7. The overload waits in the gateway's queue, not in vLLM's. Across the H100 bench the gateway queue held requests in 104 of about 2,000 five-second samples; vLLM's waiting queue was above 0 in 29 and never above 2. vLLM preempted 8 requests over the whole sweep (F41).

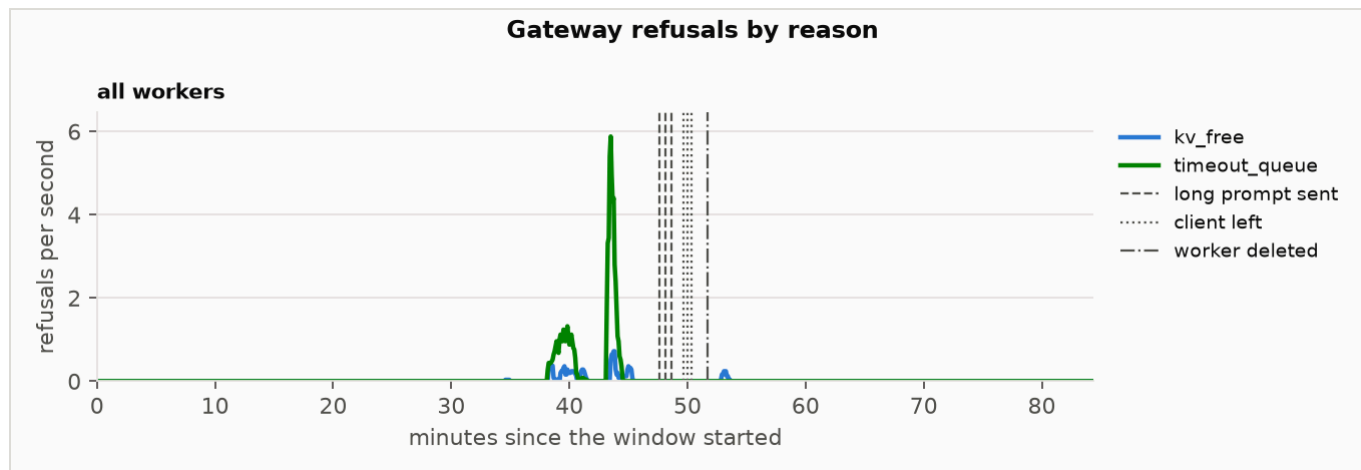


Figure 8. Refusals by reason. Once admission is sized to KV, most refusals are `timeout_queue` (a request waited past its deadline) rather than `kv_free` (F32).

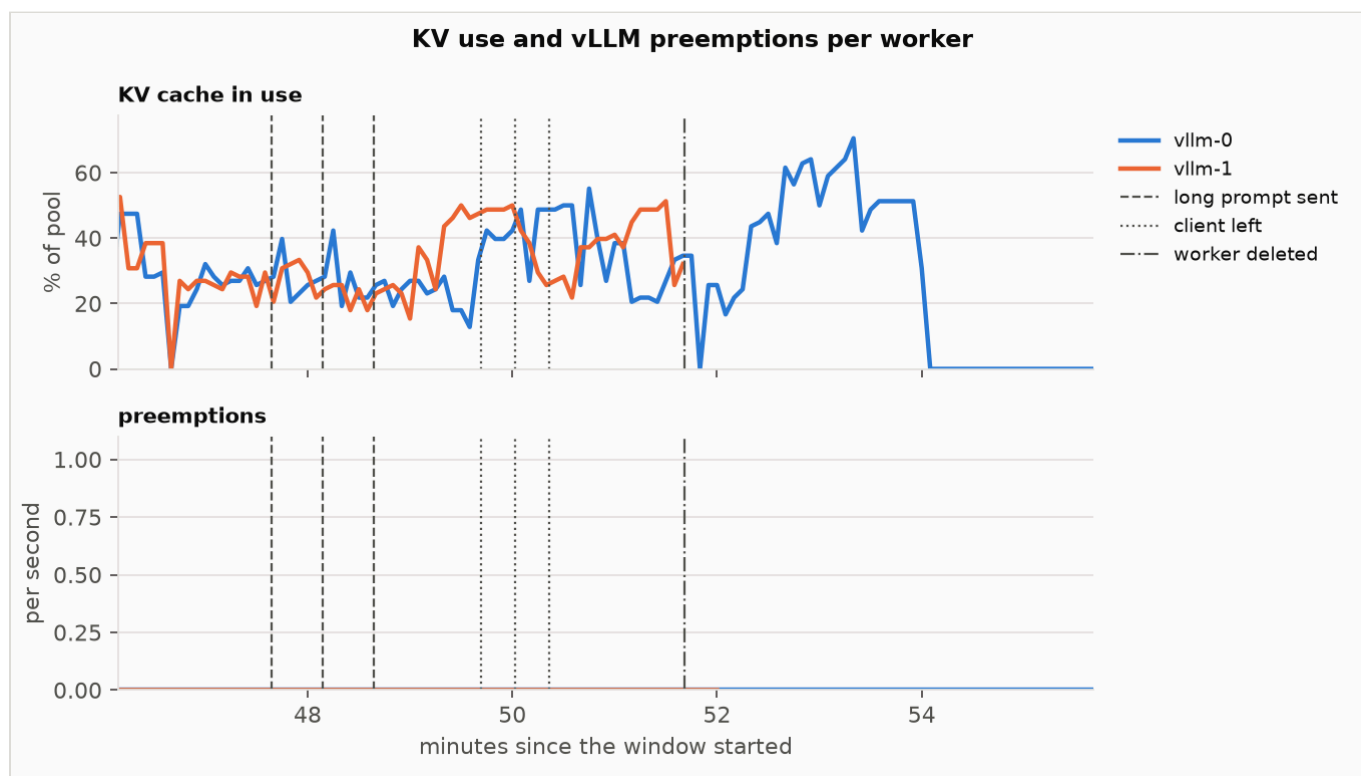


Figure 9. KV use and preemptions around the probes. The long prompts and the departed clients cause no preemption. After the worker is deleted, the remaining worker's KV rises to about 70% as it takes every run.

Probe	Result	Finding
A 10.5k-token batch prompt	Each returned in 2.1–2.5 s. The agents' inter-token latency p95 stayed at 49 ms, as in the minute before. One prompt adds two large prefill steps among many small decode steps, so a p95 can't show it.	F42
A client leaves after 3 s	vLLM's abort counter stayed at 0 on both workers. Either the cancellation doesn't reach vLLM, or vLLM 0.29 doesn't count a disconnect as an abort. The gateway logs <code>client_gone</code> but has no metric for it.	F43 (open)
A worker is deleted	The gateway marked it down 12 s later and the other worker took every placement. It was warming 5 min 35 s after deletion and ready 2 s later. Two in-flight runs ended on a 502; the agent now retries that 502.	F44

Table 8. What each probe showed.

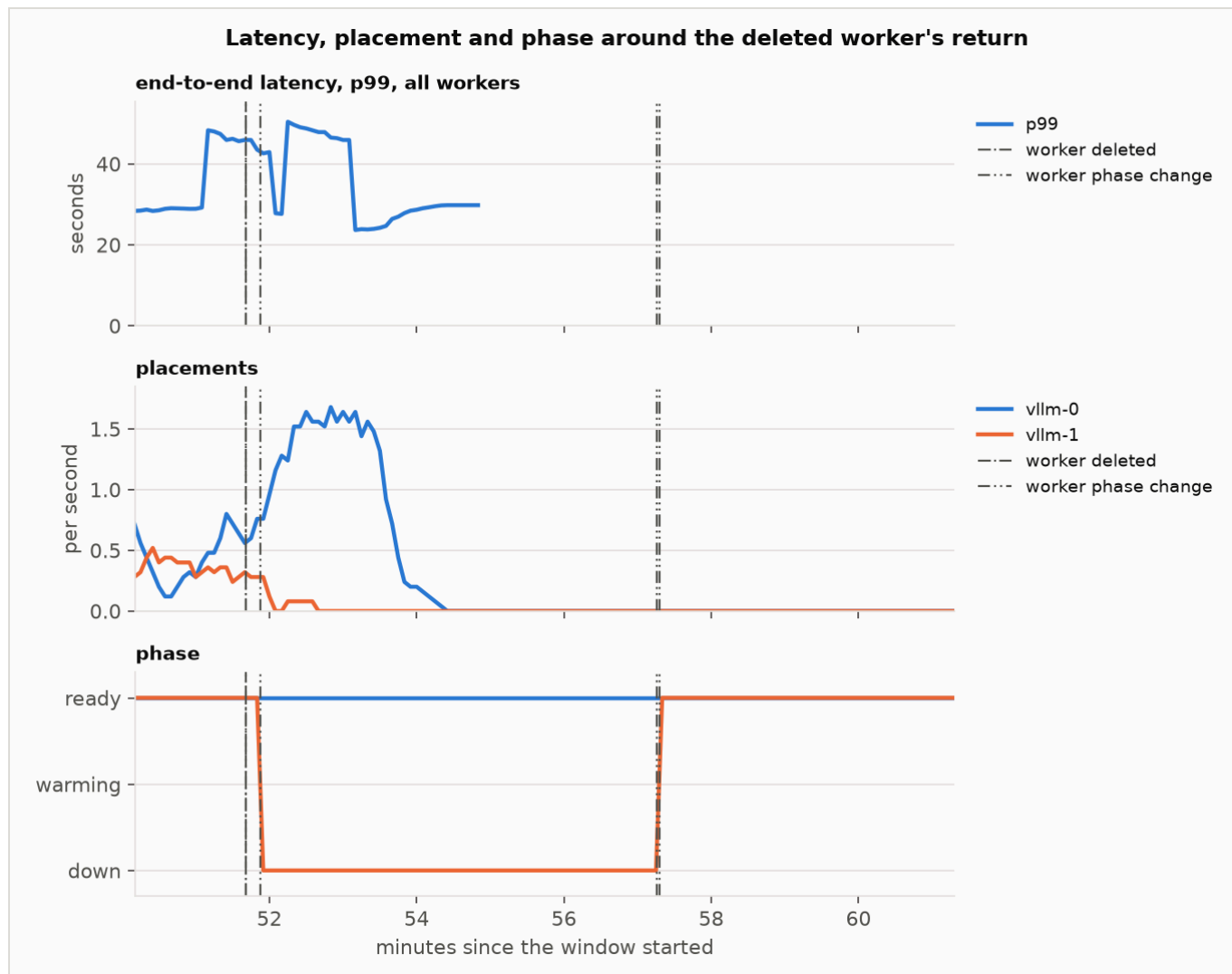


Figure 10. Around the deleted worker. Placements move to vllm-0, peaking at about 1.7 a second, while end-to-end p99 stays at 45–50 s. vllm-1 goes from down to ready in one step after its two warm-up probes. The golden run had finished before vllm-1 returned, so the ramp after a return under load is still unmeasured (F44).

Part 5. Scaling the worker pool

The vLLM pool is the only pool, and KV per worker is the limit, so KEDA adds workers rather than slots. It scales `statefulset/vllm` from 1 to 2 workers (the two halves of one H100) on two triggers, both Prometheus recording rules with promtool tests:

- **Demand:** workers wanted = $\lceil (\text{in flight} + \text{queued}) \div 4 \rceil$, the same cap admission uses, averaged over 2 minutes.
- **Capacity sheds:** any capacity refusal (`kv_free`, `queue_full`, `timeout_queue` and the like) adds a worker. A tenant over its quota doesn't count, because that isn't a capacity problem.

Run (10-07, H100)	Workers	v2 pass	What happened
as-up	1, then 2	59.6%	Concurrency 8 on one worker shed about 0.4 requests a second. The shed trigger asked for a second worker about 80 s in; it was created at 23:30:49 and ready at 23:39:10, 8 min 21 s later, after the run had ended.
as-up2	flapped	57.7%	KEDA removed the second worker 40 s into the next run, because the 10-minute window ran out before the new load reached the 2-minute averages, then re-created it.
as-up3	2	84.6%	With the window at 15 minutes (D-51) both workers stayed, and the run shed nothing. KEDA removed the second worker 15 quiet minutes after the run.

Table 9. The autoscaling session (F39). vLLM reported no errors or aborts. The runs lost on one worker ended on gateway refusals (18 in `as-up`, 16 in `as-up2`), and with two workers none did. The pass rate follows the worker count, and two workers come back near the 86.5% baseline.

The design consequence. The scaler reacts in about a minute, but a worker takes 8 minutes to load a 27B model and warm up. Scale-down must therefore wait longer than a typical gap between bursts, and the useful signal for scaling up is a capacity shed, not demand alone: under cap 4 the queue drains by shedding, so demand sat at the threshold and the demand trigger never fired (F39). The same pattern was rehearsed on kind first, with no GPU (F38).

Part 6. Validation with agent runs

Each golden task is a fault recorded from a real cluster plus an answer key the agent never sees. The checker is deterministic: nothing is judged by a model. A run passes v2 when the diagnosis names the right root object and category, states the right mechanism, cites only evidence the tools actually showed it during the run, and blames no victim or red herring (D-41).

Model, where	v2 pass [95% CI]	By tier (v2)
Qwen3-8B-AWQ, A100 slice, direct	51.9% [38.7–64.9]	easy 68, multi-hop 43, red herring 25, right-sizing 0
Qwen3-8B-AWQ, two A100 slices, through the gateway	48.1% [35.1–61.3]	easy 61, multi-hop 43, red herring 25, right-sizing 0
Qwen3-30B-A3B AWQ, whole A100	67.3% [53.8–78.5]	easy 75, multi-hop 50, red herring 75, right-sizing 50
Qwen3.8-27B-FP8, two H100 halves, through the gateway	86.5% [74.7–93.3]	easy 89, multi-hop 71, red herring 100, right-sizing 100

Table 10. Model quality on the golden set, 26 tasks run twice each (F1, F2).

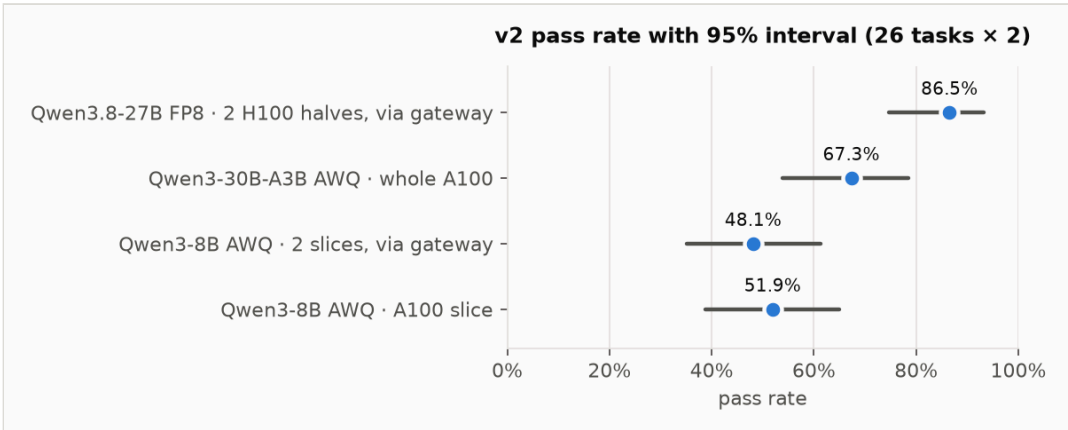


Figure 11. Qwen3.8-27B's interval sits above the 8B's. Against the 30B-A3B the intervals overlap slightly, so "better" there is likely but not established at 52 runs (F1). The gateway changes placement, not answers (F2).

Where the runs fail

- **Qwen3.8's failures are mostly the audits.** Five of its seven failed runs are the three multi-namespace audits: 22 to 24 steps, then `inconclusive` after the allowed repairs. They run out of steps rather than reasoning wrongly. The other two each passed on their repeat (F3).
- **The 8B lost points to two agent-side defects.** The `describe` tool rejected a capitalised kind 101 times, and diagnoses with more than 8 evidence refs failed validation 24 times, including runs that had found the right cause (F4).
- **Failing closed is a feature.** A diagnosis that fails validation gets two repairs, then becomes `inconclusive` instead of an ungrounded answer (D-24).

Why a model and not rules

Tier	What it tests	Rules and dashboards
easy	The pod's own status names the cause	Find these without a model
multi-hop	The symptom is on a victim and the cause elsewhere: an expired upstream certificate, an OOM-killed database, a LimitRange, a ResourceQuota	Point at the victim
red herring	The obvious suspect is healthy: a wrong CA bundle, a probe starved of CPU, a Service port, pod DNS	Point at the suspect
right-sizing	Idle against throttled against not worth changing	Give numbers, not judgement

Table 11. The tiers (D-31). Qwen3.8-27B scores 71% on multi-hop and 100% on red herrings, where the 8B scored 43% and 25%.

Part 7. Operating it: alerts, 10× traffic, open questions

Production alerts

Alert	Fires when
KVCacheSaturated	KV above the gateway's 0.80 shed line for 5 minutes
GatewayQueueWaitHigh	Queue wait p95 above 5 s, half the interactive deadline
VLLMPreempting	Any preemption for 5 minutes
DoctorInconclusiveRateHigh	Over 20% of diagnoses inconclusive in an hour
RestrictedRequestOffBox	Any restricted request leaves the box (critical)

Table 12. promtool tests check that each rule fires on its condition and stays quiet just below it (D-45). `orch_restricted_offbox_total` was 0 in all 13 gateway scrapes.

At 10× traffic

- Add workers, not bigger slices: KV per worker is the limit. On one node that means a second card; KEDA's maximum grows with it.
- Retune the in-flight cap to 6–8 per H100 half, or lengthen the interactive queue deadline (F36).
- A second gateway replica needs a shared run table, or runs partitioned by id, so stickiness survives.
- An overflow backend for non-restricted work turns refusals into slower answers.
- The KV hop pays off once contexts grow, if its transfer is confirmed.
- Settings that don't fit this workload: the 768-token output cap never bound (F30), and chunked prefill rarely does (F15).
- Watch power, not utilisation: DCGM's utilisation read 100% at any load, while power rose from about 450 W at up to 8 runs to about 600 W at 16 and 32 (F46).

Still open

Question	What would settle it
Does the KV hop move the KV, or does the destination recompute?	Force a hop and compare the hopped step's cached tokens; vLLM 0.29 has no transfer metric (F35, F45)
Does vLLM free a departed client's KV?	A gateway counter for <code>client_gone</code> , plus vLLM's running count and KV in the seconds after (F43)
What is the right cap?	Sweep caps 6 and 8 at 52 runs per level against Table 7
How does a returning worker ramp under load?	Background load that outlasts its 5.5-minute return (F44)
How does the agent do on a live cluster?	Inject faults on the node's k3s and time detection and diagnosis with <code>agent watch</code>

Table 13. Each needs a GPU session of 10 to 30 minutes.

Exhibits. Grafana screenshots

From the 10-08 H100 session, covering the golden run, the sweep from 1 to 32 concurrent runs, and the probes (about 100 minutes). Five panels have known display problems, listed in F46.

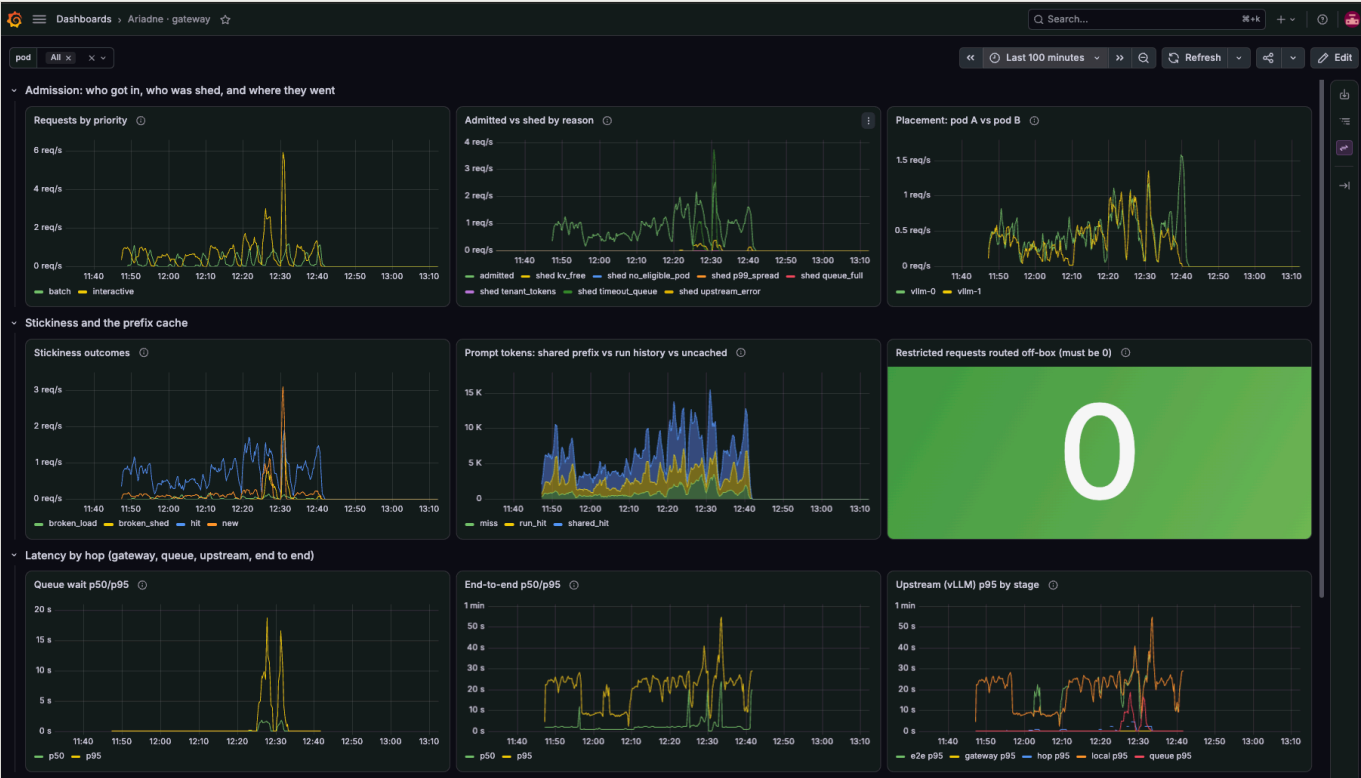


Exhibit A. Gateway: admission and latency.



Exhibit B. Gateway's view of the workers: KV free, in flight and queued, worker phase, the warm-up TTFT it re-quotes, upstream errors, KV hops by result, and responses cut at the 768-token cap. The worker phase and KV hop panels have the display problems in F46.

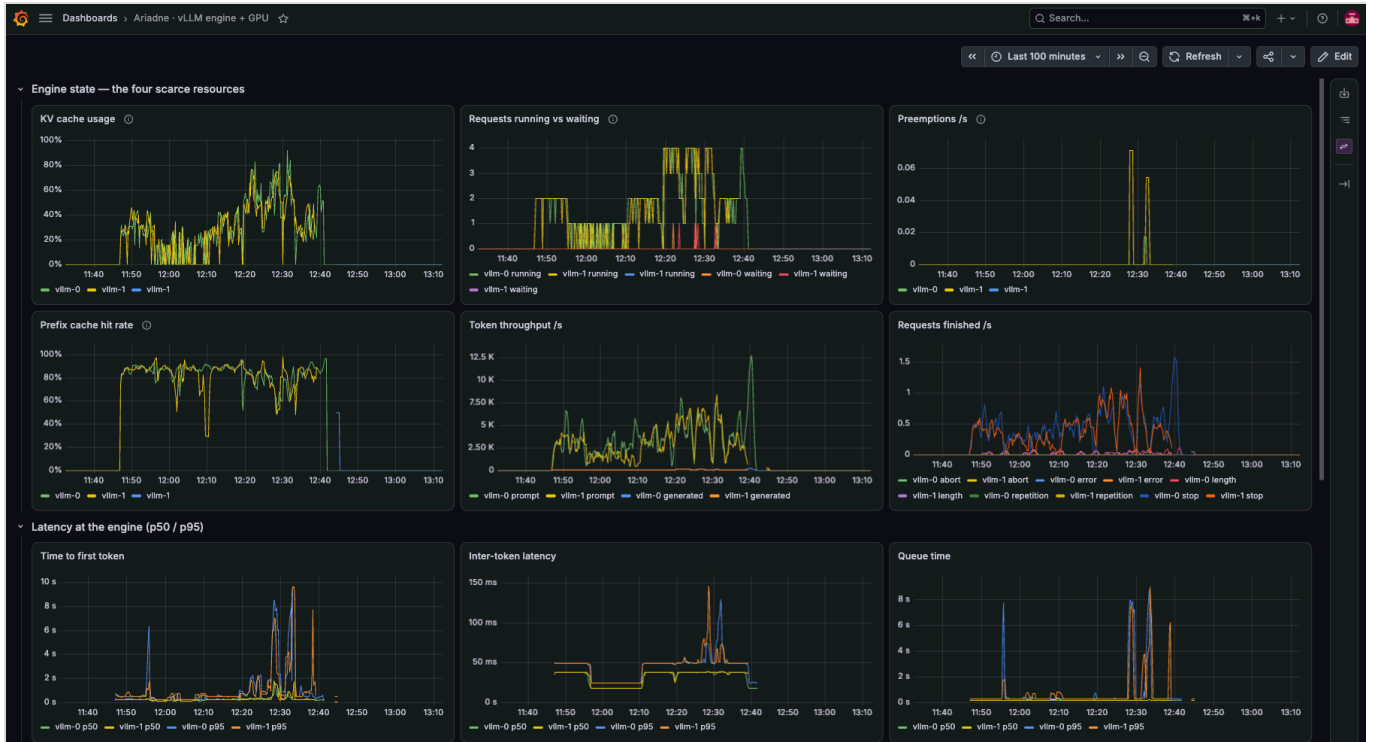


Exhibit C. vLLM: engine state and latency.



Exhibit D. vLLM end-to-end latency and prompt length, then the GPU from DCGM. Utilisation reads 100% at any load, while power tracks it: about 450 W, rising toward 600 W at the high concurrency levels, and about 110 W idle (F46). The dip in framebuffer at 12:39 is the deleted worker.

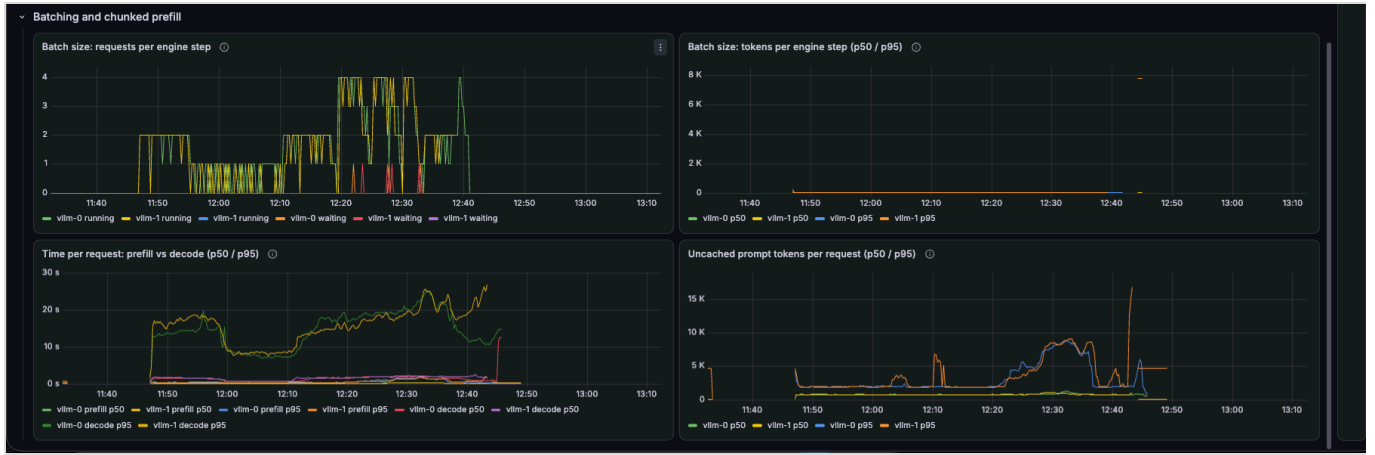


Exhibit E. vLLM: batching.

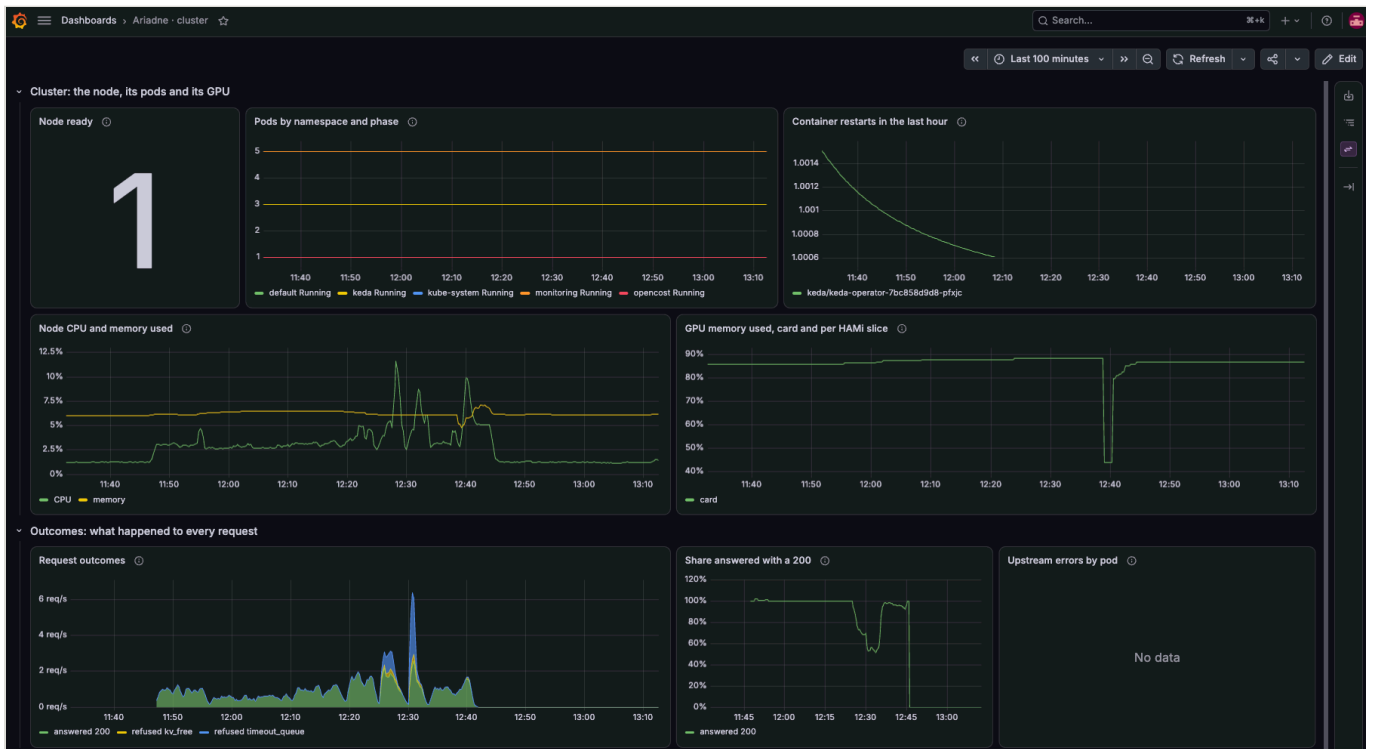


Exhibit F. Cluster: the node, its pods, GPU memory, and every request's outcome by reason. The refusal spikes around 12:25–12:32 are the 16 and 32 concurrency levels.



Exhibit G. Cluster: scaling.

Appendix. Reproducing the runs

Offline, no GPU

```
uv sync && make lint test      # Python, Go and the alert rules; CI runs the same
make report                    # rebuild the notebook and figures from metrics/
make matrix                    # regenerate the model matrix (CI fails if stale)
# break a local kind cluster on purpose
make lab-up && make inject FAULTS=oom
```

A GPU session

```
make preflight && make up      # lint + tests + fit gate, then the first GPU with capacity
make bringup                  # deploy → scale → measure KV → gateway → dashboards
make tunnel                    # terminal 2: the gateway at localhost:8000
make grafana                   # terminal 3: Grafana at localhost:3000
make check                    # both workers ready through the gateway, KV hop on or off
make bench TAG=q1-h100         # golden set + sweep 1→32 + metrics + probes + export
# the routing A/B's control arm
make gateway POLICY=least_loaded && make bench TAG=q1-h100-ll
git add metrics && git commit  # the node's Prometheus keeps nothing
make down                      # billing stops
```

Where the evidence is

Path	What
metrics/golden-*.jsonl, *.summary.json	One row per agent run (every step's latency, tokens, refusals, tool calls, the diagnosis and its score), and the summary per run tag
metrics/{gateway,vllm-0,vllm-1}-*.prom	Prometheus scrapes taken at the end of each level
metrics/ts-*.json, events-*.jsonl	Time series for the bench window, and the probe times
report/report.ipynb	Every chart in this report, rebuilt from metrics/
design/findings.md	F1-F47, each labelled measured, estimate, observed or unverified, with its file
design/decisions.md	D-1-D-51: what was chosen, why, and when to revisit it
evals/golden/, faults/, fixtures/	The tasks and answer keys, the fault manifests, and the recorded cluster output